

Effective C++

• 条款01：视C++为一个语言联邦

- C++是一个多重范式的编程语言，一个同时支持
 - 过程形式
 - 面向过程形式
 - 函数形式
 - 泛型形式
 - 元编程形式
- 为了理解C++，其主要的次语言有以下几个
 - C：说到底C++仍然是以C为基础
 - Object-Oriented C++：这也就是C with class 所诉求的，包括封装，继承，多态，virtual函数（动态绑定）等等。这一部分面向对象设计之古典守则则在C++上最直接实施
 - Template C++：C++的泛型编程部分，他们的威力强大，可以带来模板元编程
 - STL：这是一个Template 程序库，他对于容器，迭代器，算法，函数对象的规则有着极佳的紧密配合和协调，然而templates 及程序库也可以其他想法建置出来。STL有着自己特殊的办事方式，你需要遵守它的约定
- **请记住：C++高效编程守则视情况而变化，取决于你用C++的那一部分**

• 条款02：尽量以const enum inline 替换#define（宁可以编译器替换预处理器）

- 简述：
 - 宏定义是预处理阶段，会把相对应的值进行简单的替换，但是对应的名称可能不会进入记号表
 - 宏定义不会被编译器看见，万一正好bug出现在#define这里，那么后面调试起来可能会很麻烦，尤其是代码量比较大的时候
- 使用常量替换#define的时候的**两个特殊情景**
 - **情景1：定义常量指针**
 - 常量表达式通常被放在头文件中
 - **对于char-based字符串：记得书写const两次：const char* const name="xxx";**
 - 对于std::string只用写一次：const std::string name="xxx";
 - 当然一般使用string会好些
 - **情景2：class的专属常量**
 - 为了将常量的作用域限制在class内，你必须让他称为class的一个成员；从而确保此常量至多只有一份实体，你必须让他称为一个static成员


```
class GamePlayer {
private:
    static const int NumTurns = 5;    //常量声明式
    int scores[NumTurns];            //使用该常量
    ...
};

const int GamePlayer::NumTurns;    //NumTurns 的定义;
                                   //下面告诉你为什么没有给予数值
```
 - 对于这样的常量，当我们在类中声明常量声明式并表明对应的值，那么在定义的时候就不需要再设初始值，当然你可以在定义的时候设置，声明时候不设置，也无妨

```
class CostEstimate {
private:
    static const double FudgeFactor; //static class 常量声明
    ...                               //位于头文件内
};
const double                               //static class 常量定义
CostEstimate::FudgeFactor = 1.35; //位于实现文件内
```

- 注意这种所谓的in-class初始值设定也只允许对整数常量进行

• 第二种替换的#define的小方法

- **情景3:** 如果编译器**不允许上面的in-class做法**，或者不太希望使用者去读取已经设定const的地址：可以采用下面的the enum hack补偿做法

```
class GamePlayer {
private:
    enum { NumTurns = 5 }; // "the enum hack" — 令 NumTurns
                           // 成为 5 的一个记号名称.
    int scores[NumTurns]; // 这就没问题了.
    ...
};
```

- 间接：把一个属于枚举类型的数值当数组用
- 好处：**和#define类似：取地址不合法，和#define一样不会造成非必要的内存分配**
- 其他：enum hack也是模板元编程的基础技术

• 情景4: 误用#define去写宏

- 一般情况下，虽然我们能使用#define写类似函数的宏并不会带来额外的开销，但是在调用这个宏的时候可能会因为使用不当带来麻烦
- 我们可以通过使用template inline，他不仅能获得宏带来的效率以及一般函数会有的可预料行为和安全性

• 请记住：

- **对于单纯变量，以const对象或enums将#define替换之**
- **对于形似函数的宏，使用inline函数替换#define。**

• 条款03：尽可能使用const

• 概述

- const 的一件奇妙的事情是：允许你指定一个语义约束：（也就是一个“ 不该被改动的对象 ”，而编译器会强制施行这项约束
- 关键词const多才多艺，你可以用它来修饰各种你需要的东西（**回响：C++Primer中const的使用**）

• 回响：C++ Primer中const 的使用

• 顶层const

- 在auto等部分语法中，顶层const往往都是可以被忽视的
- 顶层const往往都是固定了对应对象的本身
- 比如：
 - const int a: 顶层const
 - int*const a: 顶层const

• 底层const

- 底层const往往都是不能被忽视的。
- 比如
 - const int *p: 底层const
 - const char *const p: 左边的是底层const，右边的是顶层const

- 对于指针来说，顶层或底层const取决于是否能改变对象（p）本身，能改变p为底层，不能改变p为顶层

情景1: STL迭代器与const

- STL迭代器系以指针根据塑膜出来, 所以迭代器的作用就像一个T*指针, 声明一个const的迭代器也是具有相似的地方的, 但是可能和普通指针就有点反直觉

-

```
std::vector<int> vec;
...
const std::vector<int>::iterator iter = //iter 的作用像个 T* const
    vec.begin( );
*iter = 10;                          //没问题, 改变 iter 所指物
++iter;                              //错误! iter 是 const
std::vector<int>::const_iterator cIter = //cIter 的作用像个 const T*
    vec.begin( );
*cIter = 10;                          //错误! *cIter 是 const
++cIter;                              //没问题, 改变 cIter。
```

情景2: const与函数声明时候的应用

- 令函数返回一个常量值, 往往可以降低因客户错误而造成的意外, 而又不至于放弃安全性和高效性
- 考虑下列情况

- 举例, 如果不加const将会导致 $(a*b) = c$ 这样的错误无法被找出

```
class Rational { ... };
const Rational operator* (const Rational& lhs, const Rational& rhs);
Rational a, b, c;
...
(a * b) = c;                //在 a * b 的成果上调用 operator=
if (a * b = c) ...          //喔欧, 其实是想做一个比较 (comparison) 动作!
```

- 补充: 当然, 在C++11中, 我们可以考虑使用右值。在重载运算符函数后面加上&, &&说明此运算符只能作用于左值/右值, 防止出错

情景3: const和成员函数

- 将const实施于成员函数的目的, 是为了确认该成员函数可作用于const对象上, 基于两个理由
 - 他们使得class结构比较容易理解。因为它可以得知那个函数可以改动对象内容而那个函数不行
 - 他们使得操作const对象成为可能
- 我们需要注意: 两个成员函数如果只是常量性不同 (即函数有边有没有带const), 可以被重载。这是一个重要的C++特性 (**回响: 在C++11中, 我们使用引用限定符也一样能区分函数是否重载, 并能和const一起使用**)

```
class TextBlock {
public:
    ...
    const char& operator[] (std::size_t position) const //operator[] for
    { return text[position]; }                        //const 对象.
    char& operator[] (std::size_t position)             //operator[] for
    { return text[position]; }                          //non-const 对象.
private:
    std::string text;
};
```

TextBlock 的 operator[]s 可被这么使用:

```
TextBlock tb("Hello");
std::cout << tb[0];                //调用 non-const TextBlock::operator[]

const TextBlock ctb("World");
std::cout << ctb[0];                //调用 const TextBlock::operator[]
```

- 回响: 在C++11中, 我们使用引用限定符也一样能区分函数是否重载, 并能和const一起使用**

- 通过增加引用限定符, 方法与使用const成员函数相同, &代表只可调用左值, &&代表只可调用右值, 引用限定符必须跟在const限定符之后

- 和成员函数可以根据是否右const来区分重载版本，引用限定符也可以区分重载版本，而且能和const一起使用
- 对于有两个或者两个以上的具有相同名字的和相同参数的列表，引用限定要么都不加，要么都加上

• 情景4: bitwise constness和logical constness

- 简述：
 - bitwise constness: const成员函数不可以更改对象内任何non-static成员变量，我们的编译器就遵循的是bitwise constness
 - logical constness: 一个const成员函数可以修改它所处理对象内的某些bits，但只有客户端侦测不出的情况下的以如此。
- 虽然我们的编译器是bitwise constness，但是可能会导致logical constness的事情例如下面这个例子
 - 举例：你创建了一个常量对象并设以某值，而且只对他调用const成员函数，但是你终究还是改变了它的值


```
class CTextBlock {
public:
    ...
    char& operator[](std::size_t position) const // bitwise const 声明,
    { return pText[position]; }                // 但其实不适当.
private:
    char* pText;
};

const CTextBlock cctb("Hello"); //声明一个常量对象。
char* pc = &cctb[0];           //调用 const operator[]取得一个指针,
                                // 指向 cctb 的数据。

*pc = 'J';                      //cctb 现在有了 "Jello" 这样的内容。
```
 - 一个更改了“指针所指物”的成员函数虽然不能被叫做const，但是如果只有指针而非所指隶属物属于对象，那么此函数为bitwise const不会引发编译器争议，将导致反直观结果

• 情景5: bitwise constness 非我所欲

- 对于需要将传入部分const函数的值进行修改？
 - 我们可以考虑使用mutable，忽略编译器的bitwise constness的特性
- 举例

```
class CTextBlock {
public:
    ...
    std::size_t length() const;
private:
    char* pText;
    std::size_t textLength; //最近一次计算的文本区块长度。
    bool lengthIsValid;     //目前的长度是否有效。
};

std::size_t CTextBlock::length() const
{
    if (!lengthIsValid) {
        textLength = std::strlen(pText); //错误！在 const 成员函数内
        lengthIsValid = true;           // 不能赋值给 textLength
    }                                    // 和 lengthIsValid。
    return textLength;
}
```

```

class CTextBlock {
public:
    ...
    std::size_t length() const;
private:
    char* pText;
    mutable std::size_t textLength;           //这些成员变量可能总是
    mutable bool lengthIsValid;              //会被更改，即使在
};                                           //const 成员函数内。
std::size_t CTextBlock::length() const
{
    if (!lengthIsValid) {
        textLength = std::strlen(pText);    //现在，可以这样，
        lengthIsValid = true;               //也可以这样。
    }
    return textLength;
}

```

• 情景6：将const于非const的函数合并实现

- 当我们使用const和非const的时候，函数的语句会有大量的重复，我们需要将他们合并，进行常量性转除

```

class TextBlock {
public:
    ...
    const char& operator[](std::size_t position) const //一如既往
    {
        ...
        ...
        return text[position];
    }
    char& operator[](std::size_t position) //现在只调用 const op[]
    {
        return
            const_cast<char&>(                //将 op[] 返回值的 const 转除
                static_cast<const TextBlock&>(*this) //为*this 加上 const
                [position]                        //调用 const op[]
            );
    }
    ...
};

```

- 注意进行转型的时候，我们必须让非const版本调用const版本，反过来不行。

• 请记住：

- 将某些东西声明为const 可帮助编译器侦测出来错误的用法，const可以用于被施加于任何作用域内的对象，函数参数，函数返回类型，成员函数本体
- 编译器使用bitwise constness，但是你编写程序的时候应该使用“概念上的常量性”
- 当const和非-const成员函数有着实质等价的实现的时候，令non-const版本调用const版本可避免代码重复

• 条款04：确定对象被使用前已先被初始化

• 概述

- 关于“对象初始化这件事”，C++似乎反复无常，在某些语境下x保证被初始化为0，但是在其他语境中却不保证。
- 所以我们需要处理的办法就是：永远在使用对象之前先下手为强，将其初始化，

• 情景1：对于无任何成员的内置类型，请手动完成

- 对于无任何成员的内置类型，手动完成初始化时必要的，方式多变，你可以通过赋值，或者输入，但是一定需要完成

```

int x = 0; //对 int 进行手工初始化
const char* text = "A C-style string"; //对指针进行手工初始化
//（亦见条款3）

double d;
std::cin >> d; //以读取 input stream 的方式完成初始化.

```

• 情景2：对于内置类型以外的东西，构造函数要确保每一个对象的成员全部都初始化

• 回响：区分常规的赋值与初始化

- 赋值：对象中成员使用等于号（a=b）一般就是进行赋值
- 初始化：直接使用括号（a(b)）就是初始化
- 注意区分

- 在构造函数中，我们可以使用成员初始列替换赋值的动作，而且这么做效率高

```

ABEntry::ABEntry(const std::string& name, const std::string& address,
                 const std::list<PhoneNumber>& phones)
    :theName(name),
    theAddress(address), //现在，这些都是初始化（initializations）
    thePhones(phones),
    numTimesConsulted(0)
{ } //现在，构造函数本体不必有任何动作

```

- 同样道理，当你想要default构造一个成员变量的时候，你都可以使用成员初始列，只要指定无物作为初始化实参即可

```

ABEntry::ABEntry( )
    :theName(), //调用 theName 的 default 构造函数;
    theAddress(), //为 theAddress 做类似动作;
    thePhones(), //为 thePhones 做类似动作;
    numTimesConsulted(0) //记得将 numTimesConsulted 显式初始化为 0
{ }

```

- 补充：在C++11中，如果需要全部执行默认初始化，可以考虑调用=default

- 请在初始列中列出所有的成员并初始化，哪怕成员变量时内置类型，哪怕可能有点夸张，但是这都是值得的
- 如果成员变量中是const或references，他们就一定需要初值，不能被赋值
- 许多classes拥有多个构造函数，每个构造函数拥有自己的成员初始列，多份成员初始列的存在就会导致不受欢迎的重复和无聊的工作这种情况下
 - 可以合理的在初始列中遗漏那些“赋值表现一样好的成员变量”改用他们的赋值操作，并将那些赋值操作移往某个函数，供所有构造函数使用
 - 补充：在C++11中我们可以使用委托构造函数，将一个函数的构造委托给另外一个函数并进行初始化
- C++有着非常固定的成员初始化次序：次序总是相同
 - base class 优先于派生class
 - class成员总是以声明的次序被初始化，即使他们在成员初始列中以不同顺序出现也是如此

• 情景3：不同编译单元内定义之non-local static对象的初始化次序

- 前缀：

- static对象

- 所谓static对象，其寿命从被构造出来到程序结束为止。这种对象包括global对象，namespace对象，在classes内，在函数内，以及在file作用域内被声明为static对象。函数内static对象称为local static对象，其他static对象称为non-local static对象，程序结束时static会自动销毁，也就是他们的析构函数会在main（）结束时候自动调用

- 编译单元

- 所谓编译单元是指在产出单一目标文件的那些源码，基本上它是单一源码文件加上其所含入的头文件

- 在编译多文件时候，对于多个static对象，由于C++没有定义于不同编译单元内的non-local static对象的初始化次序并无明确定义，这就可能导致使用时候的灾难：使用未经过定义的对象

- 解决方法很简单：将这些non-local-static对象搬到给他们自己的一个自己的专属小函数里面，在函数里面声明static，返回一个引用指向它所指的对象

```

class FileSystem { ... };           //同前
FileSystem& tfs()                   //这个函数用来替换 tfs 对象；它在
{                                   //FileSystem class 中可能是个 static。
    static FileSystem fs;           //定义并初始化一个 local static 对象，
    return fs;                     //返回一个 reference 指向上述对象。
}

class Directory { ... };           //同前
Directory::Directory( params )     //同前，但原本的 reference to tfs
{                                   //现在改为 tfs()
    ...
    std::size_t disks = tfs().numDisks( );
    ...
}

Directory& tempDir()               //这个函数用来替换 tempDir 对象；
{                                   //它在 Directory class 中可能是个 static。
    static Directory td;            //定义并初始化 local static 对象，
    return td;                     //返回一个 reference 指向上述对象。
}

```

- 这个手法的好处

- 保证在“该函数被调用期间”，“首次遇上该对象定义式”初始化
- 如果没有调用对应的静态函数就不会有构造和析构成本，用哪个调那个
- 对单线程程序友好，可以先在多线程程序调用之前运行需要的所有小函数，消除多线程的“竞速形势”

• 请记住

- 为内置类型对象进行手工初始化，因为C++不保证初始化他们
- 构造函数最好使用成员初始列，而不要在构造函数本体内使用赋值操作。初值列列出的成员变量，其排列次序应该和他们在class中声明次序相同
- 为免除“跨编译单元之初始化次序”问题，请以local static对象替换non-local static对象

• 条款05：了解C++默默编写并调用了那些函数

• 情景1：对于一个“空”类，C++为你默默编写的那些函数

- 当你创建一个空类，经过C++编译器为你编译后，会放入如下的函数，所有这些函数都是public和inline的

```

class Empty {
public:
    Empty() { ... }                //default 构造函数
    Empty(const Empty& rhs) { ... } //copy 构造函数
    ~Empty() { ... }               //析构函数，是否该是
                                   //virtual 见稍后说明。
    Empty& operator=(const Empty& rhs) { ... } //copy assignment 操作符。
};

```

- 唯有这些函数被需要调用的时候他们才会被编译器创建出来

```

Empty e1;                        //default 构造函数
                                 //析构函数
Empty e2(e1);                    //copy 构造函数
e2 = e1;                         //copy assignment 操作符

```

- 注意，编译器产出的析构函数是一个non-virtual函数，除非这个class的base自生声明有virtual析构函数

• 情景2：copy函数的创建与调用

- 对于正常的成员对象，编译器会尽其所能的为他声明没有定义过的函数
- 对于已经自定义过的函数（如自定义构造函数）那么编译器就不会为他生成

- 对于stl，class等，编译器会调用已有的copy函数进行copy

对于内置类型，编译器会拷贝前一个对象的每一个bits来完成赋值

- 对于某些不符合正常情况下的赋值，编译器会拒绝为其自动生成一个copy函数，而是让作者进行定义
- 比如下面这个，编译器会直接拒绝进行赋值，因为C++不允许让引用改指不同的对象

```
template<class T>
class NamedObject {
public:
    //以下构造函数如今不再接受一个 const 名称，因为 nameValue
    //如今是个 reference-to-non-const string。先前那个 char* 构造函数
    //已经过去了，因为必须有个 string 可供指涉。
    NamedObject(std::string& name, const T& value);
    ...
private:
    std::string& nameValue; //这如今是个 reference
    const T objectValue; //这如今是个 const
};
```

现在考虑下面会发生什么事：

```
std::string newDog("Persephone");
std::string oldDog("Satch");
NamedObject<int> p(newDog, 2); //当初撰写至此，我们的狗 Persephone
//即将度过其第二个生日。
NamedObject<int> s(oldDog, 36); //我小时候养的狗 Satch 则是 36 岁，
//-- 如果她还活着。
p = s; //现在 p 的成员变量该发生什么事？
```

- **回响：三五法则**
 - 所有五个拷贝控制成员应该看作一个整体：一般来说如果一个类定义了任意一个拷贝操作，他就应该定义五个所有操作
- **请记住**
 - 编译器可以暗自为class创建default构造函数，copy构造函数，assignment操作符，析构函数
- **条款06：若不想使用编译器自动生成的函数，就应该明确拒绝**
 - 概述
 - 当你不想要一个任何一个编译器为你合成的函数的时候。在C++11中，请直接使用delete，快速且安全
 - **请记住**
 - 为驳回编译器自动提供的机能，我们可以使用delete直接明确的拒绝编译器为我们合成。

• 条款07：为多态基类声明virtual析构函数

- **回响：执行动态绑定的三个条件**
 - 有直接基类
 - 使用指针/引用指向派生类
 - 调用对应的虚函数
- **情景1：对于有派生类的基类，需要声明一个virtual析构函数**
 - 对于派生类的指针（其指向一个派生类的对象）在将其赋予其基类的指针后，但是那个对象却经过基类的指针去delete，如果你的虚函数不定义为virtual的，那么就会导致删除不干净，资源泄露
 - virtual函数的目的是允许derived class实现得以客制化。任何class只要带有virtual函数都几乎确定应该也有一个virtual析构函数
- **情景2：对于不打算用来作为基类的类，声明virtual析构函数是一个坏主意**
 - 如果class不含有virtual函数，通常表示它并不意图用作一个baseclass，当class不企图作为一个base class，令其析构函数为virtual 往往是个馊主意。
 - 原因是，在声明virtual的时候会增加一个vptr的指针，这个与virtual实现有关，使用后会使得C++中对象的体积有一定的增大，导致内存的浪费
 - 因此，只有当class内含至少一个virtual函数，才将它声明为virtual析构函数

- 补充：在C++11中，我们可以使用final 来点名：不允许基类被继承。

情景3：不要考虑去继承STL容器。

- 不要考虑继承STL容器，因为他们的析构函数都不是virtual的，如果继承了他们，在编写的时候依然会出现情景1的问题。

情景4：纯虚函数与析构函数

- 对于如果你想创建一个抽象类，但是手头没有合适的纯虚函数，那么你可以考虑把析构函数声明为一个纯虚函数。

```
class AWOV {                                //AWOV = "Abstract w/o Virtuals"
public:
    virtual ~AWOV( ) = 0;                    //声明 pure virtual 析构函数
};
```

- 这个class有一个纯虚函数，又成功的变成了抽象class，因为他的析构函数为纯虚函数，所以你不需要担心析构函数的问题，但是呢，你必须为这个纯虚函数定义一下

```
AWOV::~~AWOV( ) { }                        //pure virtual 析构函数的定义
```

- 这样，这个析构函数的运行方式就是
 - 最深层派生的拉个class其析构函数最先被调用
 - 然后是每一个base class析构函数被调用
 - 编译器会在这个AMOV的deriveclasses的析构函数创建要给对~AWOV的调用动作，如果没有的话编译器就会抱怨

情景5：总结

- 给baseclasses一个virtual的析构函数这个规则只是适用于多态性质的base classes上。他的设计目的是用来通过基类接口处理派生类对象
- 某些classes的设计目的就是用来作为base classes使用的，而不是为了多态用途，他们也不需要虚析构函数

请记住

- 带多态性质的base classes应该声明一个virtual析构函数。如果class带有任何virtual 函数，他就应该拥有一个virtual析构函数
- Classes的设计目的如果不是作为base classes使用，或不是为了具备多态性，就不该被声明为virtual 析构函数

条款08：别让异常逃离析构函数

- 概述：C++并不禁止析构函数抛出异常，但是它不鼓励你这么，这是有道理的

回顾：异常的使用方法

情景1：销毁一个vector内的成员

- 考虑这个代码

```
• 代码
class Widget {
public:
    ...
    ~Widget( ) { ... } //假设这个可能吐出一个异常
};
void doSomething()
{
    std::vector<Widget> v;
    ...
} //v 在这里被自动销毁
```

- 当假设要析构vector以内的元素的时候，widget连续两次抛出异常，
- 在多个异常的情况下，程序不是结束就是导致不明确行为

情景2：面对可能抛出异常的析构函数，我们应该怎么办

- 对于一个析构函数。如果可能抛出异常，我们由两种办法解决这个问题

- 使用try-catch，抛出异常就用abort结束程序

```
DBConn::~DBConn( )
{
    try { db.close(); }
    catch (...) {
        制作运转记录，记下对 close 的调用失败；
        std::abort( );
    }
}
```

- 吞下因为调用close而发生的异常

```
DBConn::~DBConn( )
{
    try { db.close(); }
    catch (...) {
        制作运转记录，记下对 close 的调用失败；
    }
}
```

- 我们也可以进行“双保险策略”，通过更改接口函数，保证所需要的函数可以正常运行

```
class DBConn {
public:
    ...
    void close( )                //供客户使用的新函数
    {
        db.close( );
        closed = true;
    }
    ~DBConn()
    {
        if (!closed) {
            try {
                db.close( );        //关闭连接（如果客户不那么做的话）
            }
            catch (...) {
                制作运转记录，记下对 close 的调用失败；    //如果关闭动作失败，
                                                            // 记录下来并结束程序
                                                            // 或吞下异常。
                ...
            }
        }
    }
private:
    DBConnection db;
    bool closed;
};
```

• 请记住

- 析构函数绝对不要吐出异常。如果一个被析构函数调用的函数可能抛出异常，析构函数应该能够捕捉任何异常，然后吞下他们或结束程序
- 如果客户需要对某个操作函数运行期间抛出的异常作出反应，那么class应该提供一个普通函数（而非在析构函数中）执行该操作

• 条款09：绝不要在构造和析构过程中调用virtual函数

- 概述：你不应该在构造函数和析构函数期间调用virtual函数
- 回顾：virtual函数与动态绑定的具体操作

- virtual函数，在直接基类里面我们可以将它定义为virtual函数。这种函数有以下特点
 - 我们在直接基类定义过这种函数，那么在继承类我们可以继续使用它的定义，或者将这个虚函数重新定义一个
 - 在定义的时候我们要注意两个点：
 - override：对于首个virtual函数我们不能加override，对于下一个继承类重新定义的函数我们才加override
 - virtual：virtual这个关键字我们在继承类里面可加可不加那个样子。但是记得加override保证函数不充载
 - 析构函数常常被定义为virtual的，负责进行动态的调用

- 动态绑定：
 - 动态绑定往往发生在有virtual和继承类的时候，并且通过指针调用，这样才能发生动态绑定
 - 动态绑定与静态类型的区分
 - 静态类型：指还没有开始运行的时候的变量自己本身类型
 - 动态类型：指程序开始运行之后的实际类型
 - 动态绑定：指通过静态指针在开始运行的时候，如果调用的函数是虚函数，会根据静态指针指向的实际类型进行函数的调用

情景1：在构造函数/析构函数中调用virtual函数

- 如果在构造函数中调用virtual函数，看似合理，但是请注意，在base class构造期间，virtual函数并不是virtual函数。
- 理解：因为base class构造函数执行比derived class 的构造函数执行的早，在base class成员执行期间，derived class成员变量尚未初始化，如果此时调用virtual函数下降到derived classes 的阶层是不可能的，因为那些成员变量也没有初始化
- 根本原因：在derived class对象和base class构造期间，对象的类型是base classes而不是derived class。对象derived class构造函数开始执行前不会成为一个derived class，而是base class类型
- 当然，相同的道理也适用于析构函数，一旦derived class析构函数开始执行，对象内derived class变量就会呈现未定义值，所以C++视他们仿佛不再存在，进入base class析构函数后对象就会成为一个base class对象，C++的任何部分也这么看他

情景2：套娃操作，在构造函数中调用函数，函数中调用virtual函数

- 代码

```
class Transaction {
public:
    Transaction() {
        { init(); }
        virtual void logTransaction() const = 0;
        ...
private:
    void init()
    {
        ...
        logTransaction();
    }
};
```

//调用 non-virtual...

//这里调用 virtual!

- 这种情况下编译器和链接器不会做任何抱怨，你成功的蒙骗过去了编译器
- 但是请注意，对于上图的纯虚函数，这种调用就会终止代码，但是如果不是纯虚函数，则编译器会干出和情景一样的事情，即调用基类的virtual而不是你想要的derived版本的virtual函数。
- 所以，我们需要的保证的一个事情是：不仅仅是确定你的构造函数和析构函数都没有（在对象创建和被销毁期间）调用virtual函数，而且他们调用的所有函数也都应该服从这一个约束

情景3：对于这种操作，可以用来替代的方案有哪些呢

- 代码

```

class Transaction {
public:
    explicit Transaction(const std::string& logInfo);
    void logTransaction(const std::string& logInfo) const; //如今是个
                                                         //non-virtual 函数
    ...
};

Transaction::Transaction(const std::string& logInfo)
{
    ...
    logTransaction(logInfo); //如今是个
                             //non-virtual 调用
}

class BuyTransaction: public Transaction {
public:
    BuyTransaction( parameters )
        : Transaction(createLogString( parameters )) //将 log 信息
        { ... } //传给 base class 构造函数
    ...
private:
    static std::string createLogString( parameters );
};

```

- 方法：令derived classes将必要的构造信息向上传递给base class 构造函数，替换加以弥补
- 这里需要注意的点：例子中 create logstring的运用的好处
 - 首先：利用辅助函数创建一个值传给base class构造函数往往比较方便
 - 其次，让这个函数为static，也就不可能以外指向初期未初始化的derive对象内尚未初始化的成员变量，利用这样，让base class构造和析构期间调用virtual函数不下降值derived classes

请记住

- 在构造和析构期间不要调用virtual函数，因为这类调用从不下降到derived class（比起当前执行构造函数和析构函数那层）

条款10：令operator=返回一个 reference to *this

- 概述
 - 关于赋值，你可以把它们写成连锁的形式
 - 为了实现连锁赋值，赋值操作运算符必须返回一个reference指向操作符的左侧实参。在你的class实现的时候也要遵循这个协议
 - 这个协议不仅适用于以上标准赋值形式，也适用于所有赋值相关运算（+=，-=，*=。。。）

回响：常见的重载运算符的使用

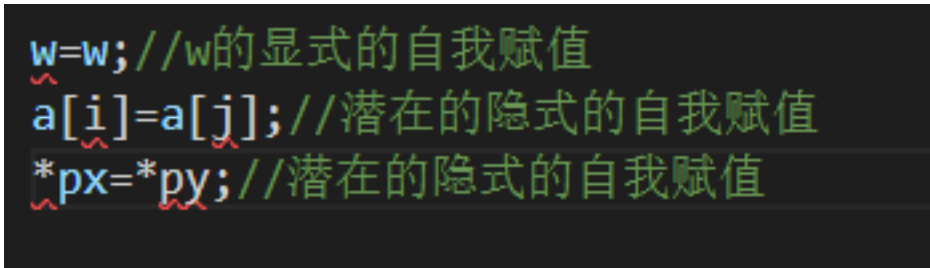
- 对于operator+，-，*，/，对于a+b，a对应this，b对应传入的对象
- 对于operator+=，-=，对于a+=b，a对应this，b对应传入的对象

请记住：

- 令赋值操作符返回一个reference to *this

条款11：在operator=中处理自我赋值

- 概述：
 - 自我赋值发生在对象赋值给自己
 - 自我赋值的几种情况



情景1: 指针指向的对象是相同的?

- 对于可能进行自赋值的两个对象，如果调用operator=可能会发生不好的知识，对于这种情况，如果出现两个相同的值就会导致赋值失败

```
Widget&
Widget::operator=(const Widget& rhs) //一份不安全的 operator= 实现版本.
{
    delete pb; //停止使用当前的 bitmap,
    pb = new Bitmap(*rhs.pb); //使用 rhs's bitmap 的副本（复印件）。
    return *this; //见条款 10。
}
```

- 一般情况下，让operator具备“异常安全性”往往会自动获得自我赋值安全的回报
- 第一种解决的办法：在复制之前，先别删除

```
Widget& Widget::operator=(const Widget& rhs)
{
    Bitmap* pOrig = pb; //记住原先的 pb
    pb = new Bitmap(*rhs.pb); //令 pb 指向 *pb 的一个复印件（副本）
    delete pOrig; //删除原先的 pb
    return *this;
}
```

- 第二种解决办法，使用所谓的copy and swap技术

```
class Widget {
...
void swap(Widget& rhs); //交换*this 和 rhs 的数据; 详见条款 29
...
};
Widget& Widget::operator=(const Widget& rhs)
{
    Widget temp(rhs); //为 rhs 数据制作一份复印件（副本）
    swap(temp); //将*this 数据和上述复印件的数据交换。
    return *this;
}
```

请记住

- 确保当对象自我赋值时，operator=有良好的行为。其中技术包括比较“来源对象”和“目标对象”的地址，精心周到的语句顺序，以及copy and swap
- 确定任何函数如果操作一个以上的对象，而其中多个对象是同一个对象时，其行为仍然正确

条款12: 复制对象时，勿忘其每一个成分

情景1: 对于自己写的copying函数，小心且负责

- copying函数：copy构造函数，copy assignment 操作符
- 如果你自己声明copying函数，意思就是告诉编译器你并不喜欢缺省实现中某些行为。编译器仿佛被冒犯死的，会以一种奇怪的方式回敬：当你实现的代码几乎必然出错时候却不告诉你
- 如果你为class添加一个成员变量，你必须同时修改copying函数，如果你忘记，编译器是不会提醒你的

情景2: 对于继承的函数，记得在继承类调用基类的copying函数

- 任何时候只要你承担起来为derived class撰写copying函数的重大责任，必须很小心的复制base class部分，而不能将任务直接交给编译器（坐视不理？）
- base class的那些成分往往都是private的，你无法直接访问他们，但是你应该让derived class 的 copying 函数调用相应的 base class函数

```

PriorityCustomer::PriorityCustomer(const PriorityCustomer& rhs)
: Customer(rhs), //调用 base class 的 copy 构造函数
  priority(rhs.priority)
{
    logCall("PriorityCustomer copy constructor");
}

PriorityCustomer&
PriorityCustomer::operator=(const PriorityCustomer& rhs)
{
    logCall("PriorityCustomer copy assignment operator");
    Customer::operator=(rhs); //对 base class 成分进行赋值动作
    priority = rhs.priority;
    return *this;
}

```

- 当你编写一个copying函数，请确保
 - 复制所有的local成员变量
 - 调用所有base classes 内适当的copying函数

• 补充

- 在行使情景二的建议的时候，你不应该
 - 令copy assignment 操作符调用copy构造函数，因为你在试图构造一个已经存在的对象。但是这根本不可行
 - 令copy构造函数调用copy assignment操作符——同样毫无意义。
- 如果你发现你的copy构造函数和copy assignment 操作符有相似的代码，消除重复代码的做法是：建立一个新的成员函数给两者调用。这样的函数往往是private的

• 请记住：

- **copying函数应该确保复制”对象内所有成员变量”以及”所有base class成分”**
- **不要尝试以某个copying函数实现另外一个copying函数，应该将共同机能放进第三个函数中，并由两个copying函数共同调用**

• 条款13：以对象管理资源

• 情景1：可能被“跳过”的delete

- 以下面这个函数为例

```

void f( )
{
    Investment* pInv = createInvestment( ); //调用 factory 函数
    ...
    delete pInv; //释放 pInv 所指对象
}

```

- 这个函数看似正确，但是呢，在。。。期间，如果因为异常等原因导致程序终止，那么就无法delete所指的对象，并且会导致资源泄露
- 为了确保资源总是被释放，我们需要将资源放进对象里面。当控制流离开f，该对象的析构函数会自动释放那些资源。实际上这正是隐身于本条款背后的半边想法：把资源放进对象里，我们可以依赖C++的析构函数自动调用机制确保资源释放
- 对于这种情况，C++11中程序库中的memory库中提供的unique_ptr满足这类情况。其析构函数对其所指对象调用delete/自己定义的对象

- **回顾：C++11中unique_ptr, share_ptr的使用方法**

-

表 12.1: shared_ptr 和 unique_ptr 都支持的操作	
shared_ptr<T> sp	空智能指针，可以指向类型为 T 的对象
unique_ptr<T> up	
p	将 p 用作一个条件判断，若 p 指向一个对象，则为 true
*p	解引用 p，获得它指向的对象
p->mem	等价于 (*p).mem
p.get()	返回 p 中保存的指针。要小心使用，若智能指针释放了其对象，返回的指针所指向的对象也就消失了
swap(p, q)	交换 p 和 q 中的指针
p.swap(q)	

表 12.2: shared_ptr 独有的操作	
make_shared<T>(args)	返回一个 shared_ptr，指向一个动态分配的类型为 T 的对象。使用 args 初始化此对象
shared_ptr<T>p(q)	p 是 shared_ptr q 的拷贝；此操作会递增 q 中的计数器。q 中的指针必须能转换为 T*（参见 4.11.2 节，第 143 页）
p = q	p 和 q 都是 shared_ptr，所保存的指针必须能相互转换。此操作会递减 p 的引用计数，递增 q 的引用计数；若 p 的引用计数变为 0，则将其管理的原内存释放
p.unique()	若 p.use_count() 为 1，返回 true；否则返回 false
p.use_count()	返回与 p 共享对象的智能指针数量；可能很慢，主要用于调试

表 12.4: unique_ptr 操作（另参见表 12.1，第 401 页）	
unique_ptr<T> u1	空 unique_ptr，可以指向类型为 T 的对象。u1 会使用 delete 来释放它的指针；u2 会使用一个类型为 D 的可调用对象来释放它的指针
unique_ptr<T, D> u2	
unique_ptr<T, D> u(d)	空 unique_ptr，指向类型为 T 的对象，用类型为 D 的对象 d 代替 delete
u = nullptr	释放 u 指向的对象，将 u 置为空
u.release()	u 放弃对指针的控制权，返回指针，并将 u 置为空
u.reset()	释放 u 指向的对象
u.reset(q)	如果提供了内置指针 q，令 u 指向这个对象；否则将 u 置为空
u.reset(nullptr)	



● **重要：以对象管理资源的两个关键想法**

- 获得资源后立刻放进管理对象内：资源取得时机便是初始化时机（RAII）
- 管理对象运用析构函数确保资源被释放

● 情景2：使用智能ptr删除“组”

- 对于share和uniqueptr，我们在使用指针要求他们删除“组”的东西的时候，我们一定要注意使用它的第二个模板参数，自定义释放函数

```
#include <iostream>
#include <string>
#include <memory>
using namespace std;
int main()
{
    string *p=new string[10]();
    shared_ptr<string> a(p,d);//d假设为一个对象，调用delete【】删除
}
```

● **请记住：**

- **为防止资源泄露，请使用RAII对象，它们在构造函数中获得资源并在析构函数中释放资源**
- **我们可以通过使用share_ptr和unique_ptr来释放资源**

● **条款14：在资源管理类小心copying行为**

- 概述
 - 我们要清楚，在资源管理类中需要注意以下几个行为
 - 防拷贝行为：

- 禁止复制：大多数的资源管理类不允许复制，复制可能不合理，在C++11中，我们需要做的就是可以使用=delete删除复制行为。unique_ptr就是一个典型的类似，他就不允许复制
- 引用计数法：有的时候我们希望保持资源，在这种情况下复制会让引用数递增，share_ptr。没错就是他
- 直接复制底层资源：我们在复制资源管理类对象的时候。需要采取深度拷贝。即直接复制资源。避免复制资源管理类
- 转移底部资源的拥有权
 - 某些罕见场合下你可能希望确保永远只有一个RAII对象指向一个未加工资源，即使RAII对象被复制依然如此。此时资源的拥有权从复制物转移到目标物。使用unique_ptr即可

请记住：

- 复制RAII对象必须一并复制它所管理的资源。所以资源的copying行为决定RAII对象的copying行为。
- 普遍而常见的RAII class copying 行为是：抑制copying（使用unique_ptr）使用引用计数法（使用share_ptr）

条款15：在资源管理类中提供对原始资源的访问

- 概述：对于指针类的对象，很多时候，如果你调用它，但是很多时候你可能需要一个底层的资源去调用然后去编写，这个时候我们需要去在资源管理类中提供对原始资源的访问

```
std::tr1::shared_ptr<Investment> pInv(createInvestment()); //见条款 13
int daysHeld(const Investment* pi);                      //返回投资天数
int days = daysHeld(pInv);                               //错误！
```

- 对于shareptr和uniqueptr，都有对应返回指针的类型，允许他们转换为底部的指针
- 很多时候，我们一般采用显式的转换掉用，保证资源管理的合理性。如果采用隐式的调用可能会引发一些其他不想看到的隐式的调用

请记住

- API往往要求访问原始资源，所以每一个RAIIclass 应该提供一个取得其所管理之资源的办法
- 对原始资源的访问可能经由显式转换或隐式转换。一般而言显式转换比较安全，隐式转换对客户比较方便

条款16：成对使用new和delete时候要采取相同形式

情景1：指向一个数组的指针和一个普通的指针

- 对于这样的指针，我们需要区分：数组调用删除数组的形式，指针调用删除指针的形式

```
std::string* stringPtr1 = new std::string;
std::string* stringPtr2 = new std::string[100];
...
delete stringPtr1;                //删除一个对象
delete [ ] stringPtr2;            //删除一个由对象组成的数组
```

- 如果你对着指向数组的指针调用了指向普通指针类型的删除模式，那么则只会删除指针指向的变量，剩下的变量将会泄露出去

情景2：对于使用typedef等的“别名”充满警惕

- 很多时候如果调用了typedef去定义，可能我们会让一个指向数组的指针看起来像是一个指向常量的指针

```
typedef std::string AddressLines[4]; //每个人的地址有 4 行，
//每行是一个 string
```

由于 AddressLines 是个数组，如果这样使用 new:

```
std::string* pal = new AddressLines; //注意, "new AddressLines" 返回
//一个 string*, 就像
//"new string[4]" 一样。
```

- 这个时候我们必须清楚，调用delete需要使用删除数组的指针，尽管它长得像一个普通的变量。为了避免出现这种错误，使用引用是一个不错的方法

```
delete pal;           //行为未有定义!
delete [ ] pal;       //很好。
```

- **请记住:**

- 如果你在new表达式中使用【】，必须在相应的delete表达式中也是用【】。如果你在new表达式中不使用【】，一定不要在相应的delete表达式中使用【】。

- **条款17：以独立语句将newed对象置入智能指针**

- 概述

- 在重新获取对象，以对象管理资源的时候，我们必须以单独的语句将对象放置于智能指针里面，如果合并其他语句，可能会导致因为语句的先后问题发生错误或者资源泄露
- 我们必须将其拆开，保证他没有“重新排列的自由”，让他按照我们所需的语句进行排列

- 将这样的语句

```
processWidget(std::tr1::shared_ptr<Widget>(new Widget), priority());
```

- 改为这样的

```
std::tr1::shared_ptr<Widget> pw(new Widget);    //在单独语句内以
                                                // 智能指针存储
                                                // newed 所得对象。
processWidget(pw, priority());                  //这个调用动作绝不至于造成泄漏。
```

- 在“资源被创建”和“资源被转换为管理对象”两个时间点之间有可能发生异常干扰

- **请记住:**

- 以独立语句将newed对象存储与智能指针内，如果不这样做，一旦异常被抛出，有可能导致难以察觉的资源泄露

- **条款18：让接口容易被正确使用，而不是被误用**

- 概述

- 很多时候，如果客户使用了某个接口，却没有获得他所预期的行为，那么这个错误设计者也有一份责任
- 如果客户企图使用某个接口而却没有获得他所预期的行为，这个代码不应该通过编译，如果代码通过了编译，那么他就是客户想要的

- **情景1：使用类型系统进行初始变量的管理**

- 很多时候，我们可能会因为手误导致输入错误

-

```
class Date {
public:
    Date(int month, int day, int year);
    ...
};
Date d(30, 3, 1995);    //喔欧! 应该是 "3, 30" 而不是 "30, 3"
```

第二，他们可能传递一个无效的月份或天数：

```
Date d(2, 30, 1995);    //喔欧! 应该是 "3, 30" 而不是 "2, 30"
```

- 很多时候，我们可以通过对于将成员变量变为一个新的“类类型”并为他设计合适的外覆类型来设置，然后再使用下，可以有效的避免上述的错误

```
struct Day {
explicit Day(int d) : val(d) { }
int val;
};

struct Month {
explicit Month(int m) : val(m) { }
int val;
};

struct Year {
explicit Year(int y) : val(y) { }
int val;
};

class Date {
public:
    Date(const Month& m, const Day& d, const Year& y);
    ...
};
Date d(30, 3, 1995);           //错误!不正确的类型
Date d(Day(30), Month(3), Year(1995)); //错误!不正确的类型
Date d(Month(3), Day(30), Year(1995)); //OK, 类型正确
```

● 情景2：对于单一外覆类型的管理

- 首先，对于上面这种，我们要遵循条款2：不要使用enums，因为这没有类型安全性，取而代之我们应该使用一个函数去返回
- 同时我们需要遵循条款4：使用一个静态函数来返回
- 其次我们使用条款三：可以用const等限定，确保因为用户限定类型而犯错
- 最后我们还要遵循一个一般性的原则：让我们定义的运算符等等的行为符合内置类型的行为。
- 最后我们才能写出这样的代码

```
class Month {
public:
    static Month Jan() { return Month(1); } //函数，返回有效月份。
    static Month Feb() { return Month(2); } //稍后解释为什么。
    ... //这些是函数而非对象。
    static Month Dec() { return Month(12); }
    ... //其他成员函数
private:
    explicit Month(int m); //阻止生成新的月份。
    ... //这是月份专属数据。
};

Date d(Month::Mar(), Day(30), Year(1995));
```

● 情景3：如何设计接口？

- 原则1：提供行为一致的接口，减少调用者的使用负担
 - 让我们定义的运算符等等的行为符合内置类型的行为，这个根本的理由就是，提供行为一致的接口，减少调用者的使用负担
 - 这种一致性质更能让接口被方便的使用，STL就是基于这样的原则
- 原则2：先发制人
 - 为了防止出错，我们很多时候就要先发制人，比如说，对于指针的操作，我们可以尽可能使用share_ptr，甚至是就不用普通ptr，将资源管理的责任全部推给share_ptr
 - 或者说，我们可能不能直接对那个普通的指针动刀子。而且不直接使用delete而是使用一个自己定义的函数，这种情况下，一个可以的方法是定义一个null-share_ptr，让其指向它，然后代替他，最后进行合适的操作

```
shared_ptr<int> p(nullptr,d); //d假设为一个自定义的删除器
p=... //指向正确对象
```

- 使用share_ptr也可以避免所谓的cross DLL problem（在A处创建在B处删除）。share_ptr可以根据习惯自动在A处创建在B处删除
- share_ptr的开销本身比裸指针大，但是在大部分场景，对其提供的安全性和方便性远远大于他的开销，特殊的场合，资源极度苛刻的时候，才不是用share_ptr。

● 请记住：

- 好的接口很容易被正确的使用，不容易被误用。你应该在所有的接口中努力达成这些性质
- 促进正确使用的办法包括接口的一致性，以及与内置类型的行为兼容。
- "阻止误用"的方法包括建立新的类型，限制类型上的操作，束缚对象值，以及消除客户的资源管理责任
- share_ptr支持定制形态删除器，可防范DLL问题，可以被用来自动解除互斥锁等等

● 条款19：设计class犹如设计type

- 我们身为C++程序员，在设计class时候，必须注意以下几条
 - 新type的对象如何创建和销毁？
 - 对象初始化和对象赋值该有什么样的差异？
 - 新type对象如果被pass by value（以值传递），意味着什么？

- 什么是新type的“合法值”
- 你的新的type需要配合某一个继承图系嘛？
- 你的新type需要什么样的转换？
- 什么样的操作符和函数对此新type而言是合理的？
- 什么样的标准函数应该驳回？
- 谁该取用新type的成员
- 什么是新type的“未声明接口”？
- 你的新type有多么一般化？
- 你真的需要一个新type嘛？

请记住：

- Class的设计就是type的设计，在定义一个新的type之前，请确定已经考虑过本条款覆盖的所有讨论主题

条款20：宁以pass by reference to const

情景1：传值调用的时候的额外的消耗

- 考虑这样一个情况

```
class Person {
public:
    Person();
    virtual ~Person();
    ...
private:
    std::string name;
    std::string address;
};

class Student: public Person {
public:
    Student();
    ~Student();
    ...
private:
    std::string schoolName;
    std::string schoolAddress;
};
```

```
bool validateStudent(Student s);           //函数以 by value 方式接受学生
Student plato;                             //柏拉图,苏格拉底的学生
bool platoIsOK = validateStudent(plato);    //调用函数
```

- 如果传入一个值，在运行时候需要进行六次构造六次析构，但是传入引用就不会造成任何额外的负担
- 所以我们要记住，传入一个带有const的引用往往很多时候是最优的选择，对于是否实际需要传入一个值，需要具体的考虑。

```
bool validateStudent(const Student& s);
```

情景2：利用传入引用避免切割问题

- 考虑这样一个情况

```
class Window {
public:
    ...
    std::string name() const;           //返回窗口名称
    virtual void display() const;       //显示窗口和其内容
};

class WindowWithScrollBars: public Window {
public:
    ...
    virtual void display() const;
};

void printNameAndDisplay(Window w)      //不正确! 参数可能被切割。
{
    std::cout << w.name();
    w.display();
}
```

- 这个时候如果进行传值，通过拷贝构造函数，你传入的w会被切割，但是如果是const引用，就可以确保不被切割，这对调用virtual函数的时候尤其重要

```
void printNameAndDisplay(const Window& w) //很好,参数不会被切割
{
    std::cout << w.name();
    w.display();
}
```

补充

- 对于C++的底层。reference底层是用指针的方式实现的，通过引用传递的其实是指针，传递一个引用会被传递一个值高效很多，还可以帮助他们免受切割的问题
- 但是也不能说传值是完全错误的，比如说STL容器很多都是传入的值，这个要按照具体使用那一部分来决定
- 可以这么说，对于pass by value可行的一般有两个：内置类型，STL，其他的在使用的时候需要好好考虑为什么这么做，尽力用pass by reference to const替换 pass by value

请记住

- 尽量以pass by reference to const替换pass by value。前者通常比较高效，并且可以避免切割问题
- 以上规则并不适用于内置类型，以及STL的迭代器和函数对象，对他们而言，pass by value往往比较合适

条款21：必须返回对象别妄想返回其reference

情景1：请不要返回一个local变量的reference

- 对于一个返回值，一旦返回一个local变量的reference，都是不可取的，因为在其作用域结束的时候，原来那个变量已经被销毁了

```
const Rational& operator* (const Rational& lhs,
                           const Rational& rhs)
{
    Rational result(lhs.n * rhs.n, lhs.d * rhs.d); //警告！糟糕的代码！
    return result;
}
```

- 如果你创建一个指针，则你无法对你创建出来的对象进行销毁，会造成资源泄露

```
const Rational& operator* (const Rational& lhs,
                           const Rational& rhs)
{
    Rational* result = new Rational(lhs.n * rhs.n, lhs.d * rhs.d); //警告！更糟的写法
    return *result;
}
```

- 就算是使用static也不行，因为static只有一个，很多时候进行比较就会出错。

```
bool operator==(const Rational& lhs, //一个针对Rationals
                const Rational& rhs); // 而写的operator==
Rational a, b, c, d;
...
if ((a * b) == (c * d)) {
    当乘积相等时，做适当的相应动作;
} else {
    当乘积不等时，做适当的相应动作;
}
```

- 所以我们应该允许函数返回一个新的对象

- 请记住：绝不要返回pointer或reference指向一个local stack对象，或返回reference指向一个heap-allocated对象，或返回pointer或reference指向一个local static对象而有可能同时需要多个这样的对象。条款4已经为“在单线程环境中合理返回 reference指向一个local static对象”提供了一份设计实例。

条款22：将成员变量声明为private

情景1：对于public成员变量，我们应该为他统一形式

- 对于public成员变量，我们应该为他统一形式：全部以函数的形式出现
- 好处1：可以让使用者都带上小括号，不用考虑是否应该带小括号这样的无聊的问题。
- 好处2：可以实现对于私有变量的针对性控制，就像电脑里面的文件一样，有些需要只读，有些不能访问，又有些需要只写，还有需要读和写一起的
- 好处3：实现封装。
 - 从封装的角度看，其实只有两种访问权限：private（提供封装）和其他（不提供封装），所以protect并不比public更加具有封装性质

情景2：对于private和protect 我们要放成员变量

- public更多的就是充当函数接口的左右。我们一般在private去放置成员变量为所有的可能的实现提供弹性
- 好处

- 在程序改变时候通知其他对象
- 验证class的约束条件，以及函数的前提和事后的状态
- 在多线程环境中执行同步控制

- **请记住:**

- 切记将成员变量声明为private，这可赋予客户访问数据的一致性，可细微划分访问控制，允诺约束条件获得保证，并提供class作者以充分的实现弹性
- protected并不比public更加具有封装性

- **条款23：宁以non-member non-friend 替换**

- **情景1：尽可能不要将同时调用类的成员函数的函数放在同一个类中**

- 考虑以下情况

```
class WebBrowser {
public:
    ...
    void clearCache( );
    void clearHistory( );
    void removeCookies( );
    ...
};
```

- 然后下面那种调用更好

```
class WebBrowser {
public:
    ...
    void clearEverything( );           //调用 clearCache,clearHistory,
                                      //和 removeCookies
    ...
};

void clearBrowser(WebBrowser& wb)
{
    wb.clearCache( );
    wb.clearHistory( );
    wb.removeCookies( );
}
```

- 对于这种情况，我们一般更加倾向于第二种调用，因为这种调用让类有了更高的封装性，更大的弹性

- 从封装的角度看：

- 如果某些东西被封装，他就不在可见，越多的东西被封装，越少的人可以看到它，越少的人看到他，我们就可以有越大的弹性变化他，因为我们只是改变了直接影响看到改变的那些人和事物。
- 越多的东西被封装，我们改变那些东西的能力也就愈大。这就是我们首先推崇封装的原因：它使我们能够改变的事物而只影响有限客户
- 对于封装性，我们可以用“有多少代码能看到一块数据”来测量。越多的函数可以访问它，数据的封装性越低

- **情景2：对于上述这种情况，如果我们需要表明他们这样类似的关系，可以把他们放入同一个命名空间里面**

- **回响：命名空间的使用方法**

-

```
#include <iostream>
namespace namen{
    void func();
    class funccc{};
}

void namen::func(){
    ;
}
class namen::funccc
{
private:
    /* data */
public:
    funccc(/* args */);
    ~funccc();
};
```

- 如图

```
namespace WebBrowserStuff {
    class WebBrowser { ... };
    void clearBrowser(WebBrowser& wb);
    ...
}
```

- 把他们放入同一个命名空间里面，不单单是看起来自然，而且有技术上的原因
 - 首先，他能进行多文件编译，保证不会出错
 - 其次，它可以保证让不使用webbrowser的客户不取得与其有关的函数
 - 将所有便利函数放在多个头文件内但隶属于同一个命名空间，意味着客户可以轻松扩展这一组便利函数
- **请记住：**
 - **宁可拿non-member non-friend函数替换member，这样可以增加封装性，包裹弹性和机能扩充性**

• 条款24：若所有参数都需要类型转换，请为此采用non-member函数

• 情景1：需要类型转换的函数如何放置

- 考虑一个表示有理数的类

```
class Rational {
public:
    Rational(int numerator = 0,          //构造函数刻意不为 explicit;
             int denominator = 1);      //允许 int-to-Rational 隐式转换。
    int numerator() const;              //分子（numerator）和分母（denominator）
    int denominator() const;           //的访问函数（accessors）——见条款 22。
private:
    ...
};
```

• 回响：explicit的使用

- C++ Primer：指定构造函数或转换函数 (C++11起)为显式，即它不能用于隐式转换和复制初始化。
- Effective C++：被声明为explicit的构造函数通常比其 non-explicit 兄弟更受欢迎，因为它们禁止编译器执行非预期（往往也不被期望）的类型转换。除非我有一个好理由允许构造函数被用于隐式类型转换，否则我会把它声明为explicit。我鼓励你遵循相同的政策。
- 假设我们需要定义一个operator *，如果定义为成员函数内部，则只能满足两个Rational类的相乘，满足不了类与普通数字的混合运算
- 对于这种情况，有两种解决方法：
 - 将构造函数声明为explicit的
 - 把operator *拿到外面去，允许类的隐式转换
 - 问：为什么对于this对象，不能进行类的隐式转换呢？

答：因为C++规定，只有当参数被列于参数列内，这个参数才是隐式类型转换的合格参与者，

- this对象不是

- **情景2：对于函数，如果他不是member函数，那么他应该变成non-member函数，还是friend函数呢？**

- 首先，要明确一点，对于member函数的反面并不是friend函数，而是non-member函数
- 所以对于不能称为member函数还要判断它是否可以成为friend函数。

- **请记住：**

- 如果你需要为某个函数的所有参数（包括被this指针所指的那个隐喻参数）进行类型转换，那么这个函数必须是non-member函数

- **条款25：考虑写出一个不抛出异常的swap函数**

- **情景1：如何交换一个对象private类里面的指针**

- 首先，对于这种交换，我们不能直接使用std::swap函数。
- 我们应该让这个类声明一个swap的public成员函数，做真正的置换工作，然后将std::swap特化，令它调用该成员函数

```
class Widget {                                //与前同，唯一差别是增加 swap 函数
public:
    ...
    void swap(Widget& other)
    {
        using std::swap;                      //这个声明之所以必要，稍后解释。
        swap(pImpl, other.pImpl);           //若要置换 Widgets 就置换其 pImpl 指针。
    }
    ...
};

namespace std {
    template<>                                //修订后的 std::swap 特化版本
    void swap<Widget>( Widget& a,
                       Widget& b )
    {
        a.swap(b);                          //若要置换 Widgets，调用其
    }                                         //swap 成员函数。
}
```

- 注意，这里的swap不能写成std::swap,因为这样写是在要求编译器强制调用std的swap函数，而不会调用特例化的swap，相反，如果按图上面写的话是让编译器从 std::swap 和特例化的swap里面选一个，当然编译器会选择特例化版本的，这样才符合题意，所以在任何情况下，我们正好要以using声明来代替直接调用std::swap

- **情景2：如果交换一个template class private的指针**

- 如果是一个template class，使用类似的方法就不行了，因为我们尝试偏特化一个函数。但是C++只允许偏特化 class。

```
namespace std {
    template<typename T>
    void swap< Widget<T> >(Widget<T>& a,      //错误！ 不合法！
                           Widget<T>& b)
    { a.swap(b); }
}
```

- **回响：template的偏特化与全特化**

- 全特化：就是将未知的typename全部替换为已知的类型
- 偏特化：就是将部分的typename替换为已知的类型
 - C++只允许class的偏特化，不允许函数的偏特化
- 我们应该在非std命名空间的namespace里面定义一个函数，以及一个swap函数的non-member版本，让他调用member函数

```

namespace WidgetStuff {
    ...                               //模板化的 WidgetImpl 等等。

    template<typename T>               //同前，内含 swap 成员函数。
    class Widget { ... };

    ...

    template<typename T>               //non-member swap 函数;
    void swap(Widget<T>& a,            //这里并不属于 std 命名空间。
              Widget<T>& b)
    {
        a.swap(b);
    }
}

```

- 注意这里不能在std空间中直接定义重载的swap版本，因为这会让操作产生未知行为。但是我们可以选择对std中的template进行特例化

• 情景3: 成员版swap不应该抛出异常

- 成员版swap函数绝对不应该抛出异常，因为swap的一个最好的应用是帮助classes提供强烈的异常安全性保障。
- swap的noexcept版本（一般的swap）不能是普通非成员版本。因为swap缺省版本是以copy构造函数和copy assignment操作符为基础，而这两者都允许抛出异常。

• 总结: 对于swap的使用

- 如果std的swap实践有着一般的效率，直接用没有问题
- 如果效率不足，那么
 - 提供一个public swap成员函数，让他高效的置换你的类型的两个对象值。
 - 在你的class或template所在的命名空间提供一个 non-member swap（一般为template特例化版本），并让他调用上述swap成员函数
 - 如果你正编写一个class（而非 class template）为你的class转化为std::swap。并令它调用你的swap成员函数。
- 最后，如果调用swap，就要确定包含using std::swap表达式，以便让std::swap在你的函数内曝光，然后不加任何namespace修饰符，赤裸裸的去调用swap。

• 请记住:

- 当std::swap对你的类型效率不高时，提供一个swap成员函数，并确定这个函数不抛出异常。
- 如果你提供一个member swap，也该提供一个non-member swap用来调用前者。对于classes（而非 templates），也请特化std::swap。
- 调用swap时应针对std::swap使用using声明式，然后调用swap并且不带任何“命名空间资格修饰”。
- 为“用户定义类型”进行std templates全特化是好的，但千万不要尝试在std内加入某些对std而言全新的东西。

• 条款26: 尽可能延后变量定义式的出现时间

• 情景1: 不应该出现过早的表达式

- 很多时候，有些看着可能被使用的表达式，实际上很可能并不会使用到，比如在异常抛出的情况

```

//这个函数过早定义变量 "encrypted"
std::string encryptPassword(const std::string& password)
{
    using namespace std;
    string encrypted;
    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
    }
    ...                               //必要动作，俾能将一个加密后的密码
                                     //置入变量 encrypted 内
    return encrypted;
}

```

```
//这个函数延后 "encrypted" 的定义，直到真正需要它
std::string encryptPassword(const std::string& password)
{
    using namespace std;
    if (password.length() < MinimumPasswordLength) {
        throw logic_error("Password is too short");
    }
    string encrypted;
    ...                //必要动作，俾能将一个加密后的密码
                       //置入变量 encrypted 内

    return encrypted;
}
```

- 第一个相比于第二个，因为第一个多了一个string的构造，所以很明显，如果出现异常，那么第二个函数更好。
- 同时，其实更好的事情就是，在他构造出来的时候就直接初始化它

```
//终于，这是定义并初始化 encrypted 的最佳做法
std::string encryptPassword(const std::string& password)
{
    ...                //检查长度。
    std::string encrypted(password); //通过 copy 构造函数
    ...                //定义并初始化。

    encrypt(encrypted);
    return encrypted;
}
```

- 所以，不只应该延后变量的定义，直到非得使用该变量的前一刻为止，甚至应该尝试延后这份定义直到能够给它初值实参为止。如果这样，不仅能够避免构造（和析构）非必要对象，还可以避免无意义的default构造行为。更深一层说，以“具明显意义之初值”将变量初始化，还可以附带说明变量的目的。

情景2：对于循环，我们如何判断变量是否出现的过早呢？

- 考虑以下情况，我们应该如何定义widget函数呢
 - 对于方法A：1构造1析构n赋值
 - 对于方法B：n构造n赋值
- 方法：除非A方法你知道赋值比构造加析构的成本低，而且你的代码对于效率高敏感，否则你应该用B方法

请记住：

- 尽可能延后变量定义式的出现，这样做可以增加程序的清晰度并改善程序效率

条款27：尽量少做转型动作

回顾：C++的四种转型

- 四种转型

```
const_cast<T>( expression )
dynamic_cast<T>( expression )
reinterpret_cast<T>( expression )
static_cast<T>( expression )
```

- const_cast通常被用来将对象的常量性去除。它也是唯一有此能力的C++-style转型操作符。
- dynamic_cast主要用来执行“安全向下转型”，也就是用来决定某对象是否归属继承体系中的某个类型。它是唯一无法由旧式语法执行的动作，也是唯一可能耗费重大运行成本的转型动作。
- reinterpret_cast意图执行低级转型，实际动作（及结果）可能取决于编译器，这也就表示它不可移植。例如将一个pointer to int转型为一个int。这一类转型在低级代码以外很少见。
- static_cast用来强迫隐式转换，例如将non-const对象转为const对象（，或将int转为double等等。它也可以用来执行上述多种转换的反向转换，例如将 void*指针转为 typed 指针，将pointer-to-base转为pointer-to-derived。但它无法将const转为non-const——这个只有const_cast才办得到。

情景1：显示转型与隐式转型

- 首先，我们要清楚的一点是，使用转型，一定会让编译器编写出对应的运行期间执行的代码，而不是什么都不做
- 其次，对于对象，我们很多时候不能直接粗暴的对于指针进行转型的操作，因为对于如下情况

```

class Base { ... };
class Derived: public Base { ... };
Derived d;
Base* pb = &d; //隐喻地将 Derived* 转换为 Base*

```

- 有的时候这两个指针的值并不相同，因为这种情况下会有一个偏移量在运行期被施行于Derived *指针上，用以取得正确的Base *指针值
- 因此，但以对象可能拥有一个以上的地址（例如上面这个），所以我们绝对不能尝试对一个已经多重继承的类指针进行转型

• 情景2：转型返回的是一个新的副本，而不是更改原来的对象

- *this执了解引用，这里实际做的操作是把一个SpWin对象cast成了Base对象，cast函数返回一个Base临时对象，然后这个对象执行onResize后就被析构了；而这里的本意是希望把SpWin对象看作一个Win对象

```

class Window { //base class
public:
    virtual void onResize( ) { ... } //base onResize 实现代码
    ...
};

class SpecialWindow: public Window { //derived class
public:
    virtual void onResize( ) { //derived onResize 实现代码
        static_cast<Window>(*this).onResize(); //将*this转型为 Window,
                                                //然后调用其 onResize;
                                                //这不可行!
        ... //这里进行 SpecialWindow 专属行为。
    }
    ...
};

```

- 所以对于这种情况，就是不要使用转型，直接通过作用域调用是最好的

```

class SpecialWindow: public Window {
public:
    virtual void onResize( ) {
        Window::onResize(); //调用 Window::onResize 作用于*this 身上
        ...
    }
    ...
};

```

• 情景3：解决“基类指针或引用”必须调用继承类的函数的操作

- 动态类型转换主要用在：拿着基类指针当做子类对象操作的时候。一般来说这种动态类型转换不是必须的，可以通过其他方式避免：一种是直接使用对应的子类类型指针去访问而不是基类，另一种是通过虚函数的方式，这个是最常用的通过基类指针访问子类成员函数的方式。
- 之所以需要dynamic_cast，通常是因为你想在一个你认定为derived class对象身上执行derived class操作函数，但你的手上却只有一个“指向base”的pointer或reference，你只能靠它们来处理对象。有两个一般性做法可以避免这个问题。
- 第一个：直接以容器的方式，里面存继承类的指针去管理容器即可，当然这个可能不能让你指向所有的派生类。如果需要建立多个，请使用多个容器去储存指针

```

typedef std::vector<std::tr1::shared_ptr<SpecialWindow> > VPSW;
VPSW winPtrs;
...
for ( VPSW::iterator iter = winPtrs.begin(); //这样写比较好,
      iter != winPtrs.end(); //不使用 dynamic_cast
      ++iter)
    (*iter)->blink();

```

- 第二个：构造一个缺省的同名函数，但是不写东西，利用virtual的性质去调用它

```

class Window {
public:
    virtual void blink() { }           //缺省实现代码“什么也没做”；
    ...                               //条款 34 告诉你为什么
};                                   // 缺省实现代码可能是个馊主意。

class SpecialWindow: public Window {
public:
    virtual void blink() { ... };      //在此 class 内，
    ...                               //blink 做某些事。
};

typedef std::vector<std::tr1::shared_ptr<Window> > VPW;
VPW winPtrs;                         //容器，内含指针，指向
...                                  // 所有可能的 Window 类型。
for (VPW::iterator iter = winPtrs.begin();
    iter != winPtrs.end();
    ++iter)                          //注意，这里没有
    (*iter)->blink();                 // dynamic_cast。

```

- 总之我们要尽可能避免使用转型，对于连串的dynamic cast更是要避免

请记住：

- 如果可以，尽量避免转型，特别是在注重效率的代码中避免 dynamic_casts。如果有设计需要转型动作，试着发展无需转型的替代设计。
- 如果转型是必要的，试着将它隐藏于某个函数背后。客户随后可以调用该函数，而不需将转型放进他们自己的代码内。
- 宁可使用C++-style（新式）转型，不要使用旧式转型。前者很容易辨识出来，而且也比较有着分门别类的职掌。

• 条款28：避免返回handles指向对象内部成分

• 情景1：空悬令牌

- 考虑以下情况

```

class Rectangle {
public:
    ...
    const Point& upperLeft( ) const { return pData->ulhc; }
    const Point& lowerRight( ) const { return pData->lrhc; }
    ...
};

class GUIObject { ... };
const Rectangle boundingBox(const GUIObject& obj); //以 by value 方式返回一个矩形
//条款 3 谈过为什么返回类型是 const
GUIObject* pgo; //让 pgo 指向某个 GUIObject
...
const Point* pUpperLeft = //取得一个指针指向外框左上点
    &(boundingBox(*pgo).upperLeft());

```

- 对boundingBox的调用获得一个新的、暂时的Rectangle对象。这个对象没有名称，所以我们权且称它为temp。随后 upperLeft作用于 temp 身上，返回一个reference 指向 temp 的一个内部成分，更具体地说是指向一个用以标示 temp 的Points。于是pUpperLeft指向那个Point对象。目前为止一切还好，但故事尚未结束，因为在那个语句结束之后，boundingBox的返回值，也就是我们所说的temp，将被销毁，而那间接导致temp内的Points析构。最终导致pUpperLeft指向一个不再存在的对象；也就是说一旦产出pUpperLeft的那个语句结束，pUpperLeft也就变成空悬、虚吊
- 这就是为什么函数如果“返回一个handle代表对象内部成分”总是危险的原因。不论这所谓的 handle 是个指针或迭代器或 reference，也不论这个 handle 是否为const，也不论那个返回handle的成员函数是否为 const。这里的唯一关键是，有个handle被传出去了，一旦如此你就是暴露在“handle比其所指对象更长寿”的风险下。
- 避免返回handles（包括references、指针、迭代器）指向对象内部。遵守这个条款可增加封装性，帮助 const 成员函数的行为像个 const，并将发生“虚吊号码牌”（dangling handles）的可能性降至最低。

请记住：

- 避免返回handles（包括references、指针、迭代器）指向对象内部。遵守这个条款可增

加封装性，帮助 const 成员函数的行为像个 const，并将发生“虚吊号码牌”的可能性降至最低。

• 条款29：为“异常安全”而努力是值得的

• 知识点1：异常安全函数的定义与行为

- 异常安全函数提供以下三个保证之一
 - 基本承诺：若异常被抛出，程序内的任何事物仍然保持在有效状态下
 - 强烈保证：如果异常被抛出，程序状态不改变
 - 不抛掷保证：承诺绝对不抛出异常，因为他们总是能够完成他们原先承诺的功能
- 当异常被抛出时候，带有异常安全性的函数会：
 - 不泄露任何资源
 - 不允许数据败坏

• 情景1：函数调用noexcept/throw () 就不会抛出异常吗？

- 不一定会
- 如果是noexcept/throw的函数抛出异常，那么就会导致意想不到的函数会被调用，他不能为你提供任何异常安全性保证，也不一定是可移植，高效的。

• 知识点2：一个可能为单一函数带来“异常安全”的做法

- 首要策略：不要为了表示某件事发生而改变对象状态，除非那件事情真的发生了
- 一个简单的策略：copy and swap：为你打算修改的对象（原件）做出一份副本，然后在那副本身上做一切必要修改。若有任何修改动作抛出异常，原对象仍保持未改变状态。待所有改变都成功后，再将修改过的那个副本和原对象在一个不抛出异常的操作中置换（swap）。
 - 好处：保证异常安全
 - 坏处：可能拷贝的时间难以接受（性能上说不过去）
- 这种只能保证“一个函数”有强烈安全的性质，但是对于一个类来说，只有所有函数都是异常安全的，那么才能说这个类是异常安全的
- 只有当你实在无法提供强烈保证，才能考虑基本保证

• 请记住：

- **异常安全函数即使发生异常也不会泄漏资源或允许任何数据结构败坏。这样的函数区分为三种可能的保证：基本型、强烈型、不抛异常型。**
- **“强烈保证”往往能够以 copy-and-swap 实现出来，但“强烈保证”并非对所有函数都可实现或具备现实意义。**
- **函数提供的“异常安全保证”通常最高只等于其所调用之各个函数的“异常安全保证”中的最弱者。**

• 条款30：透彻了解inlining的里里外外

• 概述

- inline一般被置于头文件内，在编译期优化函数，执行函数
- 编译器一般会拒绝太过于复杂的函数，以及virtual函数（或者调用）
- 构造函数和析构函数往往都是inlining的糟糕候选人
- 程序设计的人必须考虑inlining带来的冲击：如果使用inline，当程序升级的时候需要重新编译，而函数却不一定
- 不要忘记80-20经验法则：平均而言一个程序往往将80%的执行时间花费在20%的代码上头。这是一个重要的法则，因为它提醒你，作为一个软件开发者，你的目标是找出这可以有效增进程序整体效率的20%代码，然后将它inline或竭尽所能地将它瘦身。但除非你选对目标，否则一切都是虚功。

• 请记住：

- 将大多数inlining限制在小型、被频繁调用的函数身上。这可使日后的调试过程和二进制升级 (binary upgradability) 更容易, 也可使潜在的代码膨胀问题最小化, 使程序的速度提升机会最大化。
- 不要只因为function templates出现在头文件, 就将它们声明为inline。

• 条款31: 将文件的编译依存关系降至最低

- 概述:
 - 很多时候, 如果你对class的某个文件进行修改, 就会发现class把它全部都进行修改了, 这就是十分错误的。我们必须要把文件的声明与实现分开
- 知识点1: 使用handle class来分开声明与实现
 - 概述
 - 设计策略
 - 实现示例:
- 知识点2: 使用 Interface classes 实现声明与实现的分开
 - 这种class的目的是详细——描述derived classes的接口 (见条款34), 因此它通常不带成员变量, 也没有构造函数, 只有一个virtual析构函数以及一组pure virtual函数, 用来叙述整个接口。
 - 实现示例

```
class Person {
public:
    ...
    static std::tr1::shared_ptr<Person> //返回一个 tr1::shared_ptr, 指向
    create(const std::string& name,    //一个新的 Person, 并以给定之参数
           const Date& birthday,      //初始化。条款 18 告诉你
           const Address& addr);      //为什么返回的是 tr1::shared_ptr
    ...
};

class RealPerson: public Person {
public:
    RealPerson(const std::string& name, const Date& birthday,
               const Address& addr)
        : theName(name), theBirthDate(birthday), theAddress(addr)
    {}

    virtual ~RealPerson() { }
    std::string name() const;           //这些函数的实现码并不显示于此,
    std::string birthDate() const;      //但它们很容易想象。
    std::string address() const;

private:
    std::string theName;
    Date theBirthDate;
    Address theAddress;
};

std::tr1::shared_ptr<Person> Person::create(const std::string& name,
                                           const Date& birthday,
                                           const Address& addr)
{
    return
        std::tr1::shared_ptr<Person>(new RealPerson(name, birthday,
                                                    addr));
}

std::string name;
Date dateOfBirth;
Address address;
...
//创建一个对象, 支持 Person 接口
std::tr1::shared_ptr<Person> pp(Person::create(name, dateOfBirth,
                                                address));
...
std::cout << pp->name()                //通过 Person 的接口使用这个对象
          << " was born on "
          << pp->birthDate()
          << " and now lives at "
          << pp->address() ;
...
//当 pp 离开作用域,
//对象会被自动删除,
//见条款 13。
```

- 其他

- 当然这么做也会带来一些负担，比如虽然解除了耦合关系，但是因为多了一层间接的访问，他会让你在运行期间丧失很多速度，又为你让每个对象超额付出若干内存

请记住：

- 支持“编译依存性最小化”的一般构想是：相依赖于声明式，不要相依赖于定义式。基于此构想的两个手段是 **Handle classes**和**Interface classes**。
- 程序库头文件应该以“完全且仅有声明式”（.h）的形式存在。这种做法不论是否涉及**templates**都适用。具体的实现应该交给（.cpp）

条款32：确定你的public继承塑膜出is-a关系

- 概述：is-a关系

- public inheritance（公开继承）意味“is-a”（是一种）的关系。
- 如果你令class D（“Derived”）以public形式继承class B（“Base”），你便是告诉C++编译器（以及你的代码读者）说，每一个类型为D的对象同时也是一个类型为B的对象，反之不成立。你的意思是B比D表现出更一般化的概念，而D比B表现出更特殊化的概念。你主张“B对象可派上用场的任何地方，D对象一样可以派上用场”（译注：此即所谓Liskov Substitution Principle），因为每一个D对象都是一种（是一个）B对象。反之如果你需要一个D对象，B对象无法效劳，因为虽然每个D对象都是一个B对象，反之并不成立。

注意：

- 你在其他领域（包括数学）学习而得的直觉，在这里恐怕无法如预期般地帮助你。
- 代码通过编译并不表示就可以正确运作

请记住：

- “**public继承**”意味is-a。适用于base classes身上的每一件事情一定也适用于derived classes身上，因为每一个derived class对象也都是一个base class对象。

条款33：避免遮掩继承而来的名称

知识点1：回顾作用域的查找

- 当我们定义这样的一个作用域的带有继承的函数的時候

```
class Base {
private:
    int x;
public:
    virtual void mf1() = 0;
    virtual void mf2();
    void mf3();
    ...
};

class Derived: public Base {
public:
    virtual void mf1();
    void mf4();
    ...
};

void Derived::mf4()
{
    ...
    mf2();
    ...
}
```

- 对于mf2的查找：

- 编译器的做法是查找各作用域，看看有没有某个名为mf2的声明式。首先查找local作用域（也就是mf4覆盖的作用域），在那儿没找到任何东西名为mf2。于是查找其外围作用域，也就是class Derived覆盖的作用域。还是没找到任何东西名为mf2，于是再往外围移动，本例为base class。在那儿编译器找到一个名为mf2的东西了，于是停止查找。如果Base内还是没有mf2，查找动作便继续下去，首先找内含Base的那个namespace(s)的作用域（如果有的话），最后往global作用域找去。

情景1：继承，重载与作用域

- 考虑这样的代码，当继承和重载在一起时候，继承类的同名函数会遮盖掉基类的重载函数，就是像内外作用域一样，这无论是virtual还是不virtual都适用。

```

class Base {
private:
    int x;
public:
    virtual void mf1() = 0;
    virtual void mf1(int);
    virtual void mf2();
    void mf3();
    void mf3(double);
    ...
};

class Derived: public Base {
public:
    virtual void mf1();
    void mf3();
    void mf4();
    ...
};

```

- 因为以作用域为基础的名字遮盖规则没有改变，但是这么做违反了条款32，is-a是public继承的基石
- 解决方法：使用using声明式，让mf1，和mf3继承而来的东西重新可见

```

class Base {
private:
    int x;
public:
    virtual void mf1() = 0;
    virtual void mf1(int);
    virtual void mf2();
    void mf3();
    void mf3(double);
    ...
};

class Derived: public Base {
public:
    using Base::mf1;    //让 Base class 内名为 mf1 和 mf3 的所有东西
    using Base::mf3;    //在 Derived 作用域内都可见（并且 public）
    virtual void mf1();
    void mf3();
    void mf4();
    ...
};

```

• 请记住：

- **derived classes内的名称会遮掩base classes内的名称。在public继承下从来没有人希望如此。**
- **为了让被遮掩的名称再见天日，可使用 using 声明式**

• 条款34：区分接口继承和实现继承

• 知识点1：声明一个纯虚函数的目的是为了let derived classes只是继承函数接口

- 什么是函数接口？单纯的函数接口一般是一个纯虚函数来限定，使用纯虚函数就是在告诉对应的继承类：必须提供接口函数的实现函数，但是我不管你怎么实现他

```

class Shape {
public:
    virtual void draw( ) const = 0;
    ...
};

```

- 同时，我们其实可以为纯虚函数提供定义，就是为函数接口实现一份代码，调用的时候需要显式调用，这个可以考虑为非纯的虚函数提供一个更加平常和安全的实现

```

Shape* ps = new Shape;           //错误! Shape 是抽象的
Shape* ps1 = new Rectangle;      //没问题
ps1->draw( );                    //调用 Rectangle::draw
Shape* ps2 = new Ellipse;        //没问题
ps2->draw();                      //调用 Ellipse::draw
ps1->Shape::draw( );              //调用 Shape::draw
ps2->Shape::draw( );              //调用 Shape::draw

```

知识点2：声明非纯虚函数的目的，让derived classes继承该函数接口的抽象实现

- 虽然这么做是可以的，但是有可能让derived classes的设计者因为遗忘重新定义virtual函数导致使用了不正确的函数

```
class GameCharacter {
public:
    virtual int healthValue() const;    //返回人物的健康指数;
    ...                                //derived classes 可重新定义它。
};
```

- 我们一般可以将这两个分开，函数接口与抽象实现，利用知识点1的纯虚函数允许在对应的抽象基类有一份自己的实现这个特性，来进行定义，只要在使用的时候需要明确的指令

```
class Airplane {
public:
    virtual void fly(const Airport& destination) = 0;
    ...
};

void Airplane::fly(const Airport& destination) //pure virtual 函数实现
{
    缺省行为，将飞机飞至指定的目的地
}

class ModelA: public Airplane {
public:
    virtual void fly(const Airport& destination)
    { Airplane::fly(destination); }
    ...
};

class ModelB: public Airplane {
public:
    virtual void fly(const Airport& destination)
    { Airplane::fly(destination); }
    ...
};

class ModelC: public Airplane {
public:
    virtual void fly(const Airport& destination);
    ...
};

void ModelC::fly(const Airport& destination)
{
    将C型飞机飞至指定的目的地
}
```

- 知识点3: 声明non-virtual 函数的目的是为了令derived classes 继承函数的接口及一份强制性实现**

- non-virtual代表的意义：不变性凌驾于特异性，即他不应该在derived class中重新被定义

总结:

- 只是继承接口，或者继承接口以及其一份实现，或者是继承接口和一份强制实现。由于这些不同类型的声明意味着并不相同的事情，当你声明你的成员函数的时候，必须要谨慎的做选择

请记住

- 接口继承和实现继承不同。在public继承之下，derived classes总是继承base class的接口。
- pure virtual函数只具体指定接口继承。
- 简朴的（非纯）impure virtual函数具体指定接口继承及缺省实现继承。
- non-virtual函数具体指定接口继承以及强制性实现继承。

条款35：考虑virtual函数以外的其他选择

知识点1：Template Method模式

- 这个方法主张：virtual函数应该几乎总是private，较好的设计是保留一个指定函数（healthValue）为public的成员函数，但是并不是virtual，同时定义一个private virtual函数进行实际工作

```

class GameCharacter {
public:
    int healthValue() const           //derived classes 不重新定义它，
    {                                 //见条款 36。
        ...                          //做一些事前工作，详下。
        int retVal = doHealthValue(); //做真正的工作。
        ...                          //做一些事后工作，详下。
        return retVal;
    }
    ...
private:
    virtual int doHealthValue() const //derived classes 可重新定义它。
    {
        ...                          //缺省算法，计算健康指数。
    }
};

```

- 这种方法在设计模式中又被叫做模板方法，它基类抽象工作流，工作流的部分逻辑交给子类覆写
- 这种手法的一个优点在于上述代码注释“做一些事前工作”（非virtual的函数）和“做一些事后工作”（private virtual函数）之中。
- 虽然这种手法定义的private virtual有限制（子类可以继承并定义私有的虚函数，虽然子类不可以调用从父类继承的私有虚函数，但是可以定义它），但是这并不妨碍对于事情解决的方便
- 当然在这种方法下面也有非virtual private 函数，比如需要调用base class的对象的时候。

• 知识点2: Strategy模式

• 回响: 调用形式:

- 调用形式就是指对于一个函数，他的两个类型的组合：返回类型，输入类型（以及对应输入类型的个数）
- 简单调用形式举例

```

int f1();
//调用类型: int()
int f2(int);
//调用类型: int(int)
int f3(int,int);
//调用类型: int(int,int)
int* f4();
//调用类型: int*()
int** f5();
//调用类型: int**()
int& f6(int);
//调用类型: int&(int)
int&& f7(int);
//调用类型: int&&(int)

```

• 回响: 指向函数的指针

- 我们一般有三种常见的方法定义/使用函数指针

```

double (*p)(int);
//double 是返回类型, int是输入类型;
//代表p是一个指向输入类型是int, 返回类型是double的函数的指针, 两个若有一个不一样都会发生错误

int fun();
decltype(fun) *px;
//代表px是一个指向无输入类型, 返回类型是int的函数的指针, 两个若有一个不一样都会发生错误

int funx();
auto f()->int(*)(){
    return funx;
}
//此时f是一个函数, 他不接受输入, 返回一个指向无输入类型, 返回类型是int的函数的指针, 两个若有一个不一样都会发生错误

```

● 回响: function函数

- function函数尖括号里面接受一个调用形式的类型, 定义出来的function可以接收以这个调用形式进行返回的函数指针, 函数对象类对象 (对象中重载了operator ()), lambda等
- 常见操作

表 14.3: function 的操作	
function<T> f;	f 是一个用来存储可调用对象的空 function, 这些可调用对象的调用形式应该与函数类型 T 相同 (即 T 是 <i>retType(args)</i>)
function<T> f(nullptr);	显式地构造一个空 function
function<T> f(obj);	在 f 中存储可调用对象 obj 的副本
f	将 f 作为条件: 当 f 含有一个可调用对象时为真; 否则为假
f(args)	调用 f 中的对象, 参数是 args
定义为 function<T>的成员的类型	
result_type	该 function 类型的可调用对象返回的类型
argument_type	当 T 有一个或两个实参时定义的类型。如果 T 只有一个实参, 则 argument_type 是该类型的同义词; 如果 T 有两个实参, 则 first_argument_type 和 second_argument_type 分别代表两个实参的类型
first_argument_type	
second_argument_type	

- 实现方法1: 将virtual函数替换为“函数指针成员变量”。

```

class GameCharacter;           //前置声明 (forward declaration)
//以下函数是计算健康指数的缺省算法。
int defaultHealthCalc(const GameCharacter& gc);
class GameCharacter {
public:
    typedef int (*HealthCalcFunc)(const GameCharacter&);
    explicit GameCharacter(HealthCalcFunc hcf = defaultHealthCalc)
        : healthFunc(hcf)
    {}
    int healthValue() const
    { return healthFunc(*this); }
    ...
private:
    HealthCalcFunc healthFunc;
};

```

- 实现方法2: 以function成员变量替换virtual函数, 因而允许任何可调用物搭配一个兼容于需求的签名式 (调用形式)

```

class GameCharacter;           //如前
int defaultHealthCalc(const GameCharacter& gc); //如前
class GameCharacter {
public:
    //HealthCalcFunc 可以是任何“可调用物”(callable entity), 可被调用并接受
    //任何兼容于 GameCharacter 之物, 返回任何兼容于 int 的东西。详下。
    typedef std::tr1::function<int (const GameCharacter&)> HealthCalcFunc;

```

- 实践方法3: 将继承体系内的virtual函数替换为另一个继承体系内的virtual函数。

● 请记住:

- virtual函数的替代方案包括NVI手法及Strategy设计模式的多种形式。NVI手法自身是一个特殊形式的Template Method设计模式。
- 将机能从成员函数移到class外部函数, 带来的一个缺点是, 非成员函数无法访问class的non-public成员。
- tr1::function 对象的行为就像一般函数指针。这样的对象可接纳“与给定之目标签名式 (调用形式) 兼容”的所有可调用物。

